

PIU

PASSARALHOS APP

Documentação Técnica do Projeto

Arquitetura completa, modelo de IA, fluxo de dados
e integração entre o aplicativo mobile e a API de
detecção de aves por áudio.

VERSÃO DA API

0.2.0

DATA

Junho 2026

PLATAFORMA

Android / iOS

BRANCH

final

Sumário

1. Visão Geral do Projeto

- 1.1 Objetivo e Proposta de Valor
- 1.2 Stack Tecnológico
- 1.3 Estrutura de Pastas

2. Arquitetura do Sistema

- 2.1 Visão em Camadas
- 2.2 Comunicação Frontend ↔ Backend

3. Backend — PIU API

- 3.1 Estrutura da API
- 3.2 Banco de Dados (data.json)
- 3.3 Autenticação JWT
- 3.4 Endpoints — Referência Completa

4. Modelo de IA — BirdCNN

- 4.1 Visão Geral
- 4.2 Pré-processamento de Áudio
- 4.3 Arquitetura da Rede Neural
- 4.4 Treinamento
- 4.5 Inferência em Produção

5. Frontend — PIU App

- 5.1 Estrutura do App
- 5.2 Navegação
- 5.3 Camada de API (Axios)
- 5.4 Gerenciamento de Estado (Zustand)
- 5.5 Telas

6. Fluxo de Detecção de Aves

7. Sistema de XP e Gamificação

8. Configuração e Deploy Local

01

Visão Geral do Projeto

1.1

Objetivo e Proposta de Valor

PIU é um aplicativo mobile voltado para amantes de pássaros e birdwatchers. O usuário aponta o celular em direção ao som de uma ave, pressiona um botão e o app identifica automaticamente a espécie — em tempo real, usando uma rede neural convolucional treinada sobre áudios do banco de dados **Xeno-canto**, o maior repositório colaborativo de sons de aves do mundo.

Além da identificação, o app possui um sistema de **gamificação por XP**, coleção pessoal de registros, visualização em mapa e perfil de usuário — incentivando saídas ao campo e descoberta de novas espécies.

■ ■ ENTRADA Áudio em tempo real Gravação pelo microfone do dispositivo	■ PROCESSAMENTO BirdCNN (PyTorch) Rede neural convolucional customizada	■ SAÍDA Top-5 espécies Com probabilidades por espécie	■ GAMIFICAÇÃO XP + Nível + Título Sistema de progressão de usuário
--	---	---	--

1.2

Stack Tecnológico

Camada	Tecnologia	Função
App Mobile	React Native + Expo (TypeScript)	Interface do usuário
Navegação	React Navigation	Stack + Bottom Tabs
Estado global	Zustand	Gerenciamento de autenticação
Cliente HTTP	Axios	Requisições REST com interceptors JWT
Tokens	expo-secure-store	Keychain/Keystore nativo
API Backend	FastAPI 0.111.0 + Uvicorn	Servidor HTTP assíncrono
Linguagem backend	Python 3.12+	Lógica de negócio + IA
Validação	Pydantic v2	Schemas de request/response
Auth	python-jose + passlib + bcrypt 4.0.1	JWT + hash de senhas
IA / Modelo	PyTorch + torchaudio + librosa	Treino e inferência
Dataset	Xeno-canto	Gravações de aves do mundo todo
Banco de dados	JSON (data.json)	Persistência de usuários

1.3

Estrutura de Pastas

estrutura de pastas

```
piu/                                     ← raiz do monorepo
├── piu-api/                             ← Backend Python (FastAPI)
│   ├── app/
│   │   ├── main.py                      ← entry point + lifespan (carrega IA)
│   │   ├── core/config.py               ← variáveis de ambiente
│   │   ├── core/database.py             ← CRUD sobre data.json
│   │   ├── core/security.py             ← bcrypt + JWT
│   │   ├── routes/auth.py               ← endpoints de usuário
│   │   ├── routes/bird_detection.py     ← endpoints de IA
│   │   ├── schemas/auth.py              ← modelos Pydantic de auth
│   │   ├── schemas/bird.py              ← modelos Pydantic de aves
│   │   ├── ia/model.py                  ← definição + carregamento da BirdCNN
│   │   ├── ia/service.py                ← pipeline de inferência
│   │   ├── models/bird_model.pt         ← pesos do modelo treinado
│   │   ├── models/labels.npy            ← nomes das espécies
│   │   ├── train_xeno.py                 ← script de treinamento
│   │   └── data.json                    ← banco de usuários
│   └──
├── piu-app/                             ← Frontend React Native (Expo)
│   ├── App.tsx                          ← entry point React
│   └── src/
│       ├── navigation/                  ← Root, Auth, BottomTabs navigators
│       ├── store/authStore.ts           ← Zustand: estado de autenticação
│       ├── core/api/client.ts           ← Axios + interceptors JWT
│       ├── core/api/birdsApi.ts         ← chamadas de detecção de aves
│       └── modules/                     ← auth, Home, Collection, Profile, Map, Splash
```

02

Arquitetura do Sistema

2.1 Visão em Camadas

O projeto segue uma arquitetura cliente-servidor com separação clara entre apresentação (mobile), negócio (API) e inteligência artificial (modelo).

■ Apresentação	React Native + Expo
• Navegação: RootNavigator → AuthNavigator BottomTabsNavigator • Telas: Splash, Login, Register, Home, Collection, Map, Profile • Estado Global: Zustand (authStore) — user, isAuthenticated, login, logout • Camada de API: Axios com interceptors JWT + renovação automática de token • Armazenamento Seguro: expo-secure-store (Keychain iOS / Keystore Android)	
■■ Negócio	FastAPI — Python
• Middleware CORS — allow_origins=[*] • Rotas de Autenticação — /api/v1/auth/* • Rotas de Detecção — /api/v1/birds/* (requer JWT) • Banco de Dados — data.json gerenciado por database.py	
■ IA	BirdCNN — PyTorch
• Decodificação: PyAV/FFmpeg — WAV, MP3, M4A, FLAC, OGG, CAF • Pré-processamento: resample → 5s → Mel Spectrogram → Z-norm • BirdCNN: 4 blocos Conv + MaxPool + Dropout → FC classifier	

2.2 Comunicação Frontend ↔ Backend

fluxo de comunicação

App Mobile (React Native)

↓ POST /api/v1/birds/predict
↓ Authorization: Bearer <access_token>
↓ Content-Type: multipart/form-data

FastAPI (porta 8000)

↓ Valida JWT → decode_token()
↓ Valida arquivo (tipo + tamanho)
↓ BirdDetectionService.predict(bytes)

BirdCNN (PyTorch / RTX 4060)

↓ PyAV → mono → resample → 5s → MelSpec → Z-norm → tensor
↓ forward() → softmax → top-5 probabilidades

Resposta JSON → App

```
{ "species": "Northern Cardinal", "confidence": 0.92, "top5": [...] }
```

Base URL (dev): <http://192.168.1.229:8000/api/v1> | Base URL (prod): <https://api.piu.app/api/v1> | Timeout: 15s por requisição | Upload de áudio via fetch nativo (não Axios) para garantir boundary correto do multipart no Android.

03

Backend — PIU API

3.1 Estrutura da API

A API é construída com FastAPI e utiliza o mecanismo de **lifespan** para carregar o modelo de IA na inicialização, mantendo-o em memória durante toda a execução.

```
Python
# app/main.py — startup do servidor

@asynccontextmanager
async def lifespan(app):
    print("Carregando BirdCNN...")
    ia_service.bird_service = BirdDetectionService() # carrega pesos + labels
    yield # app roda aqui
    print("Encerrando...")
```

Singleton de modelo: A instância `BirdDetectionService` é criada uma única vez no startup e reutilizada em todas as requisições via `Depends(get_bird_service)`. Isso garante que o modelo não seja recarregado do disco a cada predição.

3.2 Banco de Dados (data.json)

O projeto usa um arquivo JSON como banco de dados — uma escolha deliberada para simplicidade de deploy. O arquivo é lido e escrito pela camada **database.py** a cada operação.

```
JSON
# Estrutura do data.json

{
  "users": [{
    "id": "uuid-v4",
    "name": "João Silva",
    "email": "joao@email.com",
    "password_hash": "$2b$12$...", // bcrypt
    "xp": 1240, "level": 2,
    "species_count": 12, "recordings_count": 18,
    "bird_records": [{
      "id": "uuid", "common_name": "Bem-te-vi",
      "scientific_name": "Pitangus sulphuratus",
      "confidence": 87, "location": "São Paulo, BR",
      "created_at": "2026-06-01T14:30:00Z"
    }]
  }]
}
```

3.3 Autenticação JWT

Token	Duração	Payload	Uso
Access Token	60 min	{"sub": user_id, "type": "access"}	Header Authorization: Bearer
Refresh Token	30 dias	{"sub": user_id, "type": "refresh"}	Corpo de POST /auth/refresh

Renovação automática: O Axios intercepta respostas 401 e renova o token via /auth/refresh. Requisições paralelas que falhem são enfileiradas (failedQueue) e reprocessadas após a renovação bem-sucedida, evitando múltiplas chamadas simultâneas.

3.4 Endpoints — Referência Completa

Autenticação e Usuário

Método	Endpoint	Auth	Descrição
POST	/api/v1/auth/register	Público	Cria nova conta. Retorna user + tokens.
POST	/api/v1/auth/login	Público	Autentica usuário. Retorna user + tokens.
POST	/api/v1/auth/refresh	Público	Renova access token via refresh token.
POST	/api/v1/auth/logout	JWT	Invalida sessão. Retorna 204.
GET	/api/v1/auth/me	JWT	Dados públicos do usuário logado.
PUT	/api/v1/auth/me	JWT	Atualiza name, username, bio, location.
GET	/api/v1/auth/me/records	JWT	Lista todos os registros de aves.
POST	/api/v1/auth/me/records	JWT	Cria novo registro + calcula XP.

Deteção de Aves (IA)

Método	Endpoint	Auth	Descrição
POST	/api/v1/birds/predict	JWT	Áudio → top-5 espécies com probabilidades.
GET	/api/v1/birds/species	JWT	Lista todas as espécies reconhecidas.
GET	/api/v1/birds/health	Público	Status: ok + device + nº de espécies.
GET	/health	Público	Health check geral da API.

04

Modelo de IA — BirdCNN

4.1 Visão Geral

O coração do app é uma **Rede Neural Convolucional (CNN) customizada** chamada **BirdCNN**, treinada do zero sobre gravações de aves do **Xeno-canto**. O sinal de áudio é primeiro transformado em um **Mel Spectrogram** — uma imagem que representa como a energia do som se distribui nas frequências ao longo do tempo — tornando o problema de classificação de áudio equivalente à classificação de imagens.

Dataset	Tipo	Entrada da rede	Hardware
Xeno-canto	CNN (do zero, sem transfer learning)	128 × H × 3 (Mel Spec como RGB)	CUDA (RTX 4060) / C

4.2 Pré-processamento de Áudio

- 1

Decodificação (PyAV / FFmpeg)
Bytes do arquivo → array NumPy float32 [-1, 1]. Suporta WAV, MP3, M4A, FLAC, OGG, CAF.
- 2

Conversão para Mono e Resample
Múltiplos canais → média (stereo→mono). Taxa de amostragem → 22.050 Hz via torchaudio.
- 3

Janela de 5 segundos
Áudios curtos recebem padding (zeros). Áudios longos são cortados pelo centro (trecho mais informativo).
- 4

Mel Spectrogram
128 bandas mel, FFT de 1024 pontos, hop de 512 amostras, frequência 50–14.000 Hz. Convertido para dB (top_db=80).
- 5

Normalização Z-score
 $\text{mel} = (\text{mel} - \text{mean}) / (\text{std} + 1\text{e-}6)$ — média zero, desvio padrão unitário.
- 6

Pseudo-RGB
Espectrograma (1, 128, H) repetido 3× → (3, 128, H), compatível com arquiteturas CNN padrão.

Parâmetro	Valor	Significado
sample_rate	22.050 Hz	Taxa de amostragem alvo
duration	5 segundos	Janela de tempo do áudio
n_mels	128	Número de bandas mel
n_fft	1024	Tamanho da janela FFT
hop_length	512	Passo entre janelas FFT
f_min / f_max	50 Hz / 14.000 Hz	Intervalo de frequências analisado

4.3 Arquitetura da Rede Neural

Bloco	Canais saída	Dropout	Propósito
Conv(3→32) × 2 + MaxPool	32	10% (espacial)	Características de baixo nível
Conv(32→64) × 2 + MaxPool	64	10% (espacial)	Padrões intermediários
Conv(64→128) × 2 + MaxPool	128	20% (espacial)	Características de alto nível
Conv(128→256) × 2 + AvgPool(4×4)	256	—	Representação compacta
FC: 4096 → 512 → ReLU	—	50%	Classificação nível 1
FC: 512 → 128 → ReLU	—	30%	Classificação nível 2
FC: 128 → N (espécies)	—	—	Logits por espécie

4.4 Treinamento

Hiperparâmetro	Valor
Otimizador	AdamW (lr=0,001, weight_decay=0,0001)
Scheduler	CosineAnnealingLR (T_max=epochs)
Epochs / Batch size	50 / 32
Early Stopping	Patience = 10 epochs
Loss Function	CrossEntropyLoss (label_smoothing=0.1)
Split	70% treino / 15% validação / 15% teste (estratificado)
Augmentação	Ruído Gaussiano + Ganho + SpecAugment (Time/Freq Masking)

4.5 Inferência em Produção

```
Python
# Fluxo de inferência - service.py

def predict(audio_bytes: bytes) -> PredictionResult:
    waveform = _load_waveform(audio_bytes)      # PyAV → tensor mono 22050Hz 5s
    spec     = _to_melspec(waveform)            # MelSpec → dB → Z-norm → (3,128,H)
    spec     = spec.unsqueeze(0).to(device)      # adiciona dim batch: (1,3,128,H)

    with torch.no_grad():
        probs = torch.softmax(model(spec), dim=1).squeeze().cpu().numpy()

    top5_idx = probs.argsort()[::-1][:5]
    top5 = [{"species": le.inverse_transform([i])[0], "probability": probs[i]}
            for i in top5_idx]

    return PredictionResult(species=top5[0]['species'],
                           confidence=top5[0]['probability'], top5=top5)
```

05

Frontend — PIU App

5.1 Estrutura do App

O app mobile usa **React Native + Expo** com TypeScript. Segue arquitetura modular — cada funcionalidade tem sua própria pasta em **src/modules/**, com screens, services e hooks locais.

5.2 Navegação

```

  árvore de navegação
RootNavigator (Stack)
  ■■■ [não autenticado] → AuthNavigator (Stack)
  ■   ■■■ LoginScreen
  ■   ■■■ RegisterScreen
  ■
  ■■■ [autenticado] → BottomTabsNavigator (Bottom Tabs)
  ■   ■■■ Tab 'Início' → HomeScreen      (gravação + IA)
  ■   ■■■ Tab 'Coleção' → CollectionScreen (histórico de aves)
  ■   ■■■ Tab 'Mapa' → MapaScreen      (visualização geográfica)
  ■   ■■■ Tab 'Perfil' → ProfileScreen   (dados + logout)
  ■
  ■■■ [startup] → SplashScreen (onboarding de 3 passos)
```

5.3 Camada de API (Axios)

- **Base URL dinâmica:** IP local em dev, URL de produção em build final.
- **Interceptor de request:** Injeta automaticamente o access token no header Authorization: Bearer.
- **Interceptor de response:** Detecta 401 e tenta renovar o token via /auth/refresh. Requisições paralelas são enfileiradas (failedQueue).
- **Upload de áudio:** Usa fetch nativo (não Axios) para garantir boundary correto do multipart/form-data no Android.

5.4 Gerenciamento de Estado (Zustand)

Campo / Ação	Tipo	Descrição
user	User null	Dados do usuário logado
isAuthenticated	boolean	Controla qual navegador exibir
isLoading	boolean	Estado de carregamento durante restoreSession
login()	async	Chama /auth/login, salva tokens, atualiza estado
register()	async	Chama /auth/register, salva tokens
logout()	async	Chama /auth/logout, limpa tokens e estado

Campo / Ação	Tipo	Descrição
restoreSession()	async	Verifica SecureStore + chama /auth/me

5.5 Telas

Tela	Funcionalidade principal
SplashScreen	Animação de abertura + onboarding 3 passos. Dispara restoreSession.
LoginScreen	Formulário email + senha. Usa authStore.login().
RegisterScreen	Cadastro (name, email, password). Usa authStore.register().
HomeScreen	Core do app — botão de microfone, grava áudio, envia para IA, exibe resultado, salva registro com XP.
CollectionScreen	Histórico de aves identificadas. Chama GET /auth/me/records.
MapaScreen	Visualização em mapa dos avistamentos registrados.
ProfileScreen	Dados do usuário: XP, nível, título, espécies, gravações, edição e logout.

06

Fluxo de Detecção de Aves

Fluxo completo — da gravação pelo usuário até o registro do avistamento:

1

Usuário pressiona e segura o botão

App solicita permissão de microfone. Se concedida, inicia `Audio.Recording.createAsync(HIGH_QUALITY)`. Gera M4A (Android) ou CAF (iOS). Timer conta os segundos. Ondas sonoras animam.

2

Usuário solta o botão

`stopAndUnloadAsync()` para a gravação. URI do arquivo temporário é obtida. Indicador muda para 'Analisando...' e spinner é exibido.

3

Envio para a API

Arquivo de áudio enviado via fetch nativo para `POST /api/v1/birds/predict` com access token JWT no header. fetch adiciona boundary correto automaticamente.

4

Validação no backend

JWT validado via `decode_token()`. Tipo e tamanho do arquivo verificados (máx 10 MB).

5

Pipeline de pré-processamento

PyAV decodifica → normalização float32 → mono → resample 22050 Hz → janela de 5s (centro) → Mel Spectrogram 128 bandas → dB → Z-score → tensor (3, 128, H).

6

Inferência na BirdCNN

Tensor entra na rede com `torch.no_grad()`. Logits → softmax → vetor de probabilidades → top-5 selecionados e retornados ordenados.

7

Resultado exibido ao usuário

JSON com species, confidence e top5 retornado ao app. Alert exibe espécie, confiança e 2ª possibilidade + XP ganho.

8

Salvo no histórico

`POST /auth/me/records` salva a identificação. Backend calcula XP (+100 espécie nova, +40 repetição), atualiza xp, level, species_count. Falhas são silenciosas.

07

Sistema de XP e Gamificação

O PIU conta com um sistema de progressão para engajar os usuários a descobrir novas espécies e registrar mais avistamentos.

Pontos de Experiência (XP)

Ação	XP Ganho	Condição
Identificação de espécie nova	+100 XP	Primeira vez que a espécie aparece nos registros
Re-identificação de espécie conhecida	+40 XP	Espécie já estava nos registros anteriores

Cálculo de Nível

```
Python
def calculate_level(xp: int) -> int:
    return max(1, (xp // 1000) + 1)

# Exemplos:
# XP 0-999      → Nível 1
# XP 1000-1999 → Nível 2
# XP N×1000     → Nível N+1
```

Métricas do Usuário

- xp** — pontos totais acumulados
- level** — nível atual calculado a partir do XP
- species_count** — número de espécies únicas identificadas
- recordings_count** — total de gravações realizadas
- rare_count** — contador de espécies raras (campo reservado para expansão)
- title** — título textual (ex: 'OBSERVADOR INICIANTE')

08

Configuração e Deploy Local

Variáveis de Ambiente (.env)

```
.env
# piu-api/.env

JWT_SECRET_KEY=chave-secreta-dev
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=30
MODEL_PATH=models/bird_model.pt
LABELS_PATH=models/labels.npy
```

Iniciar o Backend

```
bash
# Na pasta piu-api/ (com venv_gpu ativo)

source venv_gpu/Scripts/activate

# Instalar dependências
pip install -r requirements.txt

# Rodar o servidor (dev, com hot-reload)
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Documentação interativa (Swagger UI)
# http://192.168.1.229:8000/docs
```

Iniciar o App Mobile

```
bash
# Na pasta piu-app/

npm install
npx expo install expo-av expo-file-system
npx expo start

# Para Android (dispositivo físico ou emulador)
npx expo run:android
```

Treinar o Modelo

```
bash
# Colocar dataset em wavfiles/ e bird_songs_metadata.csv na raiz

python train_xeno.py

# Outputs gerados:
# output/bird_model.pt      ← pesos do melhor modelo
# output/labels.npy        ← lista de espécies
# output/training_history.png ← gráfico de loss/acc
# output/confusion_matrix.png ← matriz de confusão

# Copiar para produção
cp output/bird_model.pt models/
cp output/labels.npy models/
```

Dependências do Backend

Pacote	Versão	Função
fastapi	0.111.0	Framework web assíncrono
uvicorn	0.29.0	Servidor ASGI
pydantic[email]	2.7.1	Validação de dados + e-mail
python-jose[cryptography]	3.3.0	JWT
passlib[bcrypt]	1.7.4	Hash de senhas
bcrypt	4.0.1	Backend bcrypt (versão compatível)
python-multipart	0.0.9	Upload de arquivos multipart
torch + torchaudio	latest	Deep learning + processamento de áudio
librosa	latest	Fallback de leitura de áudio (sem FFmpeg)
av (PyAV)	latest	Decodificação de áudio via FFmpeg em produção
scikit-learn	latest	LabelEncoder para nomes das espécies

Banco de dados em produção: O arquivo data.json é lido/escrito diretamente a cada operação. Em produção com alta concorrência, substitua por SQLite ou PostgreSQL. O config.py já define DATABASE_URL para SQLite, indicando que a migração está planejada.